

Guide PostgreSQL - LogementMad

Ce document explique comment créer la base de données nécessaire au projet dans pgAdmin, puis contient le script SQL complet à copier/coller.

Élément	Valeur
Nom de la base	logement_vente_location
Tables créées	users, logements, commandes, historiques
Admin par défaut	admin@logement.mg / admin123
Fichier SQL inclus	database/BASE_DE_DONNEES_A_COPIER_PGADMIN.sql

Étapes dans pgAdmin

1. Ouvrir pgAdmin et se connecter au serveur PostgreSQL.
2. Ouvrir Query Tool sur la base postgres, puis exécuter : `CREATE DATABASE logement_vente_location;`
3. Dans pgAdmin, ouvrir la nouvelle base logement_vente_location.
4. Ouvrir Query Tool dans cette base.
5. Copier/coller tout le script SQL complet ci-dessous puis cliquer sur Execute.
6. Lancer ensuite le backend avec `npm run dev`. Le compte administrateur est admin@logement.mg / admin123.

Script de création de la base

```
CREATE DATABASE logement_vente_location;
```

Script SQL complet à copier dans la base logement_vente_location

```
-- =====
-- LOGEMENTMAD - BASE DE DONNEES POSTGRESQL
-- A copier/coller dans pgAdmin, dans la base : logement_vente_location
-- =====

CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(120) NOT NULL,
    email VARCHAR(160) UNIQUE NOT NULL,
    password TEXT NOT NULL,
    role VARCHAR(30) NOT NULL DEFAULT 'agent' CHECK (role IN ('admin', 'agent')),
    photo_profil TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS logements (
    id SERIAL PRIMARY KEY,
    titre VARCHAR(180) NOT NULL,
    prix NUMERIC(14, 2) NOT NULL CHECK (prix > 0),
    description TEXT NOT NULL,
    type VARCHAR(30) NOT NULL CHECK (type IN ('à louer', 'à vendre')),
    categorie VARCHAR(60) NOT NULL,
    image_principale TEXT,
    image_secondaire TEXT,
    image_tertiaire TEXT,
    id_utilisateur INTEGER REFERENCES users(id) ON DELETE SET NULL,
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS commandes (
    id SERIAL PRIMARY KEY,
    id_logement INTEGER REFERENCES logements(id) ON DELETE CASCADE,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    telephone VARCHAR(30) NOT NULL,
    adresse TEXT NOT NULL,
    message TEXT,
    statut VARCHAR(30) NOT NULL DEFAULT 'nouvelle' CHECK (statut IN ('nouvelle', 'traitée', 'annulée')),
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS historiques (
    id SERIAL PRIMARY KEY,
    action VARCHAR(100) NOT NULL,
    details TEXT,
    id_utilisateur INTEGER REFERENCES users(id) ON DELETE SET NULL,
    date_creation TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_logements_type ON logements(type);
CREATE INDEX IF NOT EXISTS idx_logements_categorie ON logements(categorie);
CREATE INDEX IF NOT EXISTS idx_logements_date_creation ON logements(date_creation);
CREATE INDEX IF NOT EXISTS idx_commandes_statut ON commandes(statut);
CREATE INDEX IF NOT EXISTS idx_commandes_date_creation ON commandes(date_creation);
CREATE INDEX IF NOT EXISTS idx_historiques_date_creation ON historiques(date_creation);

-- Administrateur par défaut : admin@logement.mg / admin123
-- Si pgcrypto est disponible, le mot de passe sera stocké en bcrypt.
```

```
INSERT INTO users (nom, email, password, role)
VALUES ('Administrateur', 'admin@logement.mg', crypt('admin123', gen_salt('bf', 10)), 'admin')
ON CONFLICT (email)
DO UPDATE SET
    nom = EXCLUDED.nom,
    password = EXCLUDED.password,
    role = 'admin';

INSERT INTO historiques (action, details)
VALUES ('Initialisation', 'Base de données LogementMad initialisée avec succès.')
ON CONFLICT DO NOTHING;
```